OpenNCCM

Guía del desarrollador

Cómo está hecho OpenNCCM, dónde está cada cosa, cómo se prueba y cómo se publica: lo necesario para continuar el proyecto.

Versión del software: 2.40.0
Noviembre de 2026

Contenido

1. Arquitectura
2. Mapa del código
3. Datos
4. Equipos y plataformas
5. Interfaz web
6. API y app móvil
7. Pruebas y laboratorio
8. Publicar y actualizar
9. Rendimiento

Guía del desarrollador de OpenNCCM

Esta guía es para quien continúa el desarrollo de OpenNCCM: cómo está hecho, dónde está cada cosa, cómo se prueba y cómo se publica. Está escrita para poder trabajar en el proyecto sin ayuda de nadie más: cada capítulo dice qué archivos tocar, en qué orden y cómo comprobar que funciona.

Capítulo	De qué trata
1. Arquitectura	Las piezas del sistema, cómo viaja una petición, procesos, hilos, tareas programadas y syslog
2. Mapa del código	Qué hace cada archivo de `app/`, agrupado por área
3. Datos	Base de datos, tablas, migraciones, ajustes, cifrado y directorio de datos
4. Equipos y plataformas	Perfiles, respaldo, inventario, cambios, reversión, recetas, configlets, cumplimiento; cómo agregar una plataforma
5. Interfaz web	Rutas, plantillas, `render()`, permisos, idiomas, archivos estáticos y convenciones de diseño
6. API y app móvil	API REST con token, API de la app móvil y el puente de acciones
7. Pruebas y laboratorio	Pruebas automáticas, CI, equipo simulado y el laboratorio GNS3 con los sellos «Verificada»
8. Publicar y actualizar	Versiones, CHANGELOG, etiquetas, instalador y actualización de los servidores
9. Rendimiento	Reglas para que las páginas sigan rápidas con miles de equipos, y cómo medirlo

Primeros pasos (en 10 minutos)

Requisitos: Python 3.10 o más nuevo (la CI prueba en Ubuntu 22.04 con 3.10 y en 24.04 con 3.12) y Git.

```
git clone https://github.com/Gaaraalex/opennccm.git && cd opennccm
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
export NCM_DATA_DIR=$PWD/.datos NCM_KEY_FILE=$PWD/.datos/secret.key NCM_SYSLOG_PORT=5514
mkdir -p .datos
.venv/bin/python -m app.cli create-admin admin Admin12345
.venv/bin/python -m app.cli seed            # políticas, configlets y ejemplos
.venv/bin/uvicorn app.main:app --reload --port 8000
```

Abra `http://127.0.0.1:8000` y entre con `admin` / `Admin12345`. Para tener un equipo con el que probar respaldos y cambios sin red, arranque el Cisco simulado y agréguelo con la dirección `127.0.0.1`, puerto `2222`, tipo «Cisco IOS» y usuario/contraseña `admin` / `cisco`:

```
.venv/bin/python tests/fakedev.py 2222
```

Las pruebas completas, como las corre la CI:

```
bash tests/run-tests.sh .venv/bin/python
```

Reglas del proyecto (las que más importan)

1. **Todo en español** en la interfaz, el código (comentarios y docstrings) y los mensajes. Cada texto nuevo de la interfaz se traduce en `app/locales/en.json` (la prueba `e2e_i18n` falla si falta alguno).
2. **Las actualizaciones nunca pierden datos.** La base se migra sola agregando columnas, tablas e índices (`app/db.py`); renombrar o transformar datos exige un paso de migración explícito y su prueba.
3. **Nada se ejecuta en un equipo sin que quede rastro.** Los cambios van por solicitud de cambio (aprobación, respaldo antes y después, reversión); las consultas quedan en Auditoría.
4. **Lo dudoso cuenta como cambio.** Si no se sabe si un comando solo consulta, se trata como cambio (`app/consolecore.py`).
5. **Cada arreglo lleva su prueba** y su línea en `CHANGELOG.md` con el formato `- [nuevo|mejora|arreglo|seguridad] **Título:** explicación`.
6. Un equipo real de un cliente nunca se usa para probar: se prueba con el equipo simulado o en el laboratorio (capítulo 7).

1. Arquitectura

OpenNCCM es **una sola aplicación web en Python** (FastAPI) que respalda, compara, audita y cambia la configuración de equipos de red por SSH (Netmiko/Paramiko), con una base de datos (SQLite por omisión, PostgreSQL opcional) y páginas HTML generadas en el servidor (Jinja2) con un poco de JavaScript propio, sin frameworks del lado del navegador.

```
Navegador ──HTTPS──> nginx ──HTTP──> uvicorn (1 proceso) ──> FastAPI app.main:app
                                          │                    ├─ rutas web (HTML)      app/main.py,
app/routes_ops.py, …
App móvil ──HTTPS─────────────────────────┤                    ├─ API REST y móvil      /api/…,
app/mobile*.py
                                          │                    ├─ consola web (WebSocket) app/console.py
Equipos ──syslog UDP 514─────────────────>│ receptor syslog    app/scheduler.py
                                          ├─ APScheduler (tareas programadas)          app/scheduler.py
                                          ├─ grupo de hilos (respaldos, cambios)       app/services.py
«pool»
                                          └─ SQLAlchemy ──> SQLite (WAL) o PostgreSQL  app/db.py
                                                      └──> SSH/SCP/HTTP a los equipos  app/services.py,
app/drivers.py
```

Un solo proceso

El servicio `opennccm.service` arranca **un** proceso de uvicorn (`--workers 1`). Es a propósito: el planificador de tareas, el receptor de syslog, la caché de ajustes, los trabajos de «Respaldar todos» y las sesiones de consola viven en memoria de ese proceso. Si algún día se necesitan varios procesos, esas piezas tendrían que salir a un proceso aparte (y la caché de ajustes, invalidarse entre procesos).

Dentro del proceso hay tres tipos de trabajo:

Trabajo	Dónde corre	Ejemplos
Peticiones web	hilos del servidor (las rutas son funciones normales; FastAPI las corre en su grupo de hilos)	páginas, formularios, API
Trabajo largo con equipos	`services.pool` (`ThreadPoolExecutor`, tamaño `NCM_WORKERS`, 16 por omisión)	respaldos, solicitudes de cambio, auditorías
Tareas programadas	APScheduler (`app/scheduler.py`)	respaldos programados, purga de versiones, avisos, firmware

Una ruta web **nunca** espera a un equipo: encola el trabajo en `services.pool` y vuelve enseguida; la página consulta el avance (por ejemplo «Respaldar todos» en `app/bulkbackup.py` y `static/bulkbk.js`).

Cómo viaja una petición

`app/main.py` crea `app = FastAPI(...)` y le pone, de fuera hacia dentro:

1. `_Gzip` — comprime las respuestas (menos las que llegan en vivo línea a línea, como ping o traceroute).
2. `SessionMiddleware` — cookie de sesión firmada (`opennccm`), 30 días como máximo.
3. `LangMiddleware` (`app/i18n.py`) — elige el idioma de la petición.
4. `MobileBridge` (`app/mobile_app.py`) — deja que la app móvil use las rutas web con su token (capítulo 6).

5. `security_headers` — cabeceras de seguridad (CSP, X-Frame-Options…), `no-store` en las páginas y caché larga en los estáticos con `?v=`.

Después, la ruta. Casi todas siguen el mismo patrón:

```
@app.get("/devices")
def devices(request: Request, q: str = ""):
    user = require(request)                # sin sesión → al login; sin la función (app/perms.py) → 403
    scope = device_scope(user)             # None = todos; si no, el conjunto de equipos que puede ver
    with SessionLocal() as db:             # una sesión corta de base de datos
        ...                                # consultas (ver capítulo 9 para que sean rápidas)
    return render(request, "devices.html", user, devices=devs)   # app/web.py
```

`render()` (`app/web.py`) agrega a la plantilla lo común: el usuario, los contadores de la barra lateral (`_nav_counts`), los avisos (`flash`), el tema, el idioma y las funciones globales (`icon`, `vlogo`, `_`, `_f`…).

Arranque

`lifespan()` en `app/main.py`:

1. `init_db()` crea las tablas que faltan, agrega columnas nuevas e índices nuevos (`app/db.py`).
2. Carga los paquetes de cumplimiento y los configlets nuevos de la versión (`compliance_packs`, `configlets_seed`).
3. `scheduler.start()` arranca APScheduler y el receptor de syslog.

Tareas programadas (`app/scheduler.py`)

Tarea	Cuándo
Programaciones del usuario (respaldo, auditoría, CVE, configlet)	su cron (id `sched-<id>`)
Purga de versiones viejas	03:30
Limpieza de grabaciones de consola	03:40
Imágenes .fxi de WatchGuard	03:00
Base de conocimiento: casos vencidos / correo semanal	08:00 / lunes 08:05
Licencias por vencer	08:10
Métricas del servidor (CPU, memoria, disco)	cada 60 s
Versión nueva de OpenNCCM / de firmware	cada 24 h
Solicitudes de cambio con hora (ventana de mantenimiento)	cada minuto
Corrección automática de cumplimiento (`autofix.sweep`; también después de cada auditoría con fallos)	cada 5 minutos

El cron de las programaciones pasa por `app/cronutil.py`: APScheduler numera los días de la semana distinto que cron (0 = lunes) y ahí se corrige.

Syslog: detección de cambios en tiempo real

El receptor UDP (puerto `NCM_SYSLOG_PORT`, 514) recibe los mensajes de los equipos. Si el mensaje dice que alguien cambió la configuración, busca el equipo por la IP de origen y programa un respaldo con un pequeño retraso (`_delayed_backup`), para que el cambio termine antes de leerlo. Ese respaldo queda con origen «syslog». Si el equipo tiene la reversión automática activada y el respaldo (de syslog o programado) encuentra un cambio fuera de una ventana de mantenimiento (es decir, que no vino de una solicitud aprobada), `services._auto_rollback` lo revierte con los mismos comandos de «Volver a esta versión».

Corrección automática de cumplimiento (`app/autofix.py`)

Apagada por defecto. Una política con `autofix` activado aplica sola, dentro de su ventana, las correcciones de las reglas elegidas: solo reglas con el sello «Verificada» (`app/verified.py`) cuya huella (`Rule.autofix_sig`: tipo, patrón y comandos) sigue siendo la aprobada. Va regla por regla y equipo por equipo: respaldo de antes, las mismas comprobaciones que una solicitud sin aprobación (`services.example_addresses`, `services.lockout_risks`, `impact.for_change`), `services.push_commands`, respaldo de después y, si el equipo rechazó algo o la regla sigue sin cumplirse, `services.rollback_commands` a la versión de antes. Cada intento queda en `AutoFixRun` y bloquea esa regla en ese equipo 24 h (`hold`, que «Reintentar» quita). El ajuste `autofix_paused` la detiene en todo el sistema. Las rutas `/compliance/autofix/…` piden la función crítica `autofix` (`app/perms.py`).

Directorio de datos y archivos del servidor instalado

Ruta	Qué hay
`/opt/opennccm`	el código (un clon de Git) y `.venv`
`/etc/opennccm/secret.key`	la clave de cifrado (AES-256). **Sin ella no se puede leer nada de lo guardado**
`/etc/opennccm/opennccm.env`	variables de entorno del servicio
`/var/lib/opennccm`	base SQLite, grabaciones de consola, imágenes de firmware, exportación Git, documentos
`/etc/nginx/sites-available/opennccm`	proxy HTTPS (lo genera el instalador)

Servicios de systemd: `opennccm.service` (la aplicación, sin privilegios, usuario `opennccm`), `opennccm-webupdate.path/.service` (actualizar desde la web: corre como root cuando aparece `update.request`) y `opennccm-fxi-firewall.path/.service` (abre el puerto FTP temporal de las imágenes .fxi).

Variables de entorno (`app/config.py`)

Variable	Por omisión	Para qué
`NCM_DATA_DIR`	`/var/lib/opennccm`	directorio de datos
`NCM_KEY_FILE`	`/etc/opennccm/secret.key`	clave de cifrado
`NCM_DATABASE_URL`	SQLite en el directorio de datos	p. ej. `postgresql+psycopg2://usuario:clave@host/opennccm` (instale `psycopg2-binary`)
`NCM_SYSLOG_PORT` / `NCM_SYSLOG_ENABLED`	`514` / `1`	receptor syslog

Variable	Por omisión	Para qué
`NCM_WORKERS`	`16`	respaldos y cambios a la vez (y tamaño del grupo de conexiones a la base)
`NCM_NVD_API_KEY`	vacío	clave de la API de NVD (CVE)
`NCM_FW_CHECK_NEW`	`1`	buscar firmware nuevo al agregar un equipo (las pruebas lo apagan)
`NCM_SECURE_COOKIE`	`0`	cookie solo por HTTPS
`NCM_TLS_CA`	`/etc/ssl/opennccm/ca.crt`	CA propia del instalador (se descarga desde el login)
`NCM_UPDATE_REPO`	`Gaaraalex/opennccm`	de dónde se buscan las versiones nuevas
`NCM_BACKUP_PASSPHRASE`	vacío	frase para cifrar la copia de recuperación
`NCM_SUPPORT_URL`, `NCM_FORUM_URL`, `NCM_ISSUES_URL`, `NCM_DONATE_*`	enlaces del proyecto	botones de soporte y donación

Dependencias principales (`requirements.txt`)

FastAPI + uvicorn[standard] (web y WebSocket de la consola), SQLAlchemy (base de datos), Jinja2 (plantillas), Netmiko (equipos por SSH/Telnet; trae Paramiko y scp), APScheduler 3 (tareas), cryptography (cifrado), requests (NVD, GitHub, fabricantes), pyotp + segno (verificación en dos pasos y su QR), psutil (rendimiento del servidor), ldap3 y pyjwt (Active Directory y Entra ID) y reportlab (documento de la red en PDF; el Word se arma sin librería). No se agregan dependencias sin una buena razón: tienen que instalarse también en Python 3.10 (Ubuntu 22.04).

2. Mapa del código

Qué hace cada archivo de `app/`. La descripción es la primera frase del docstring del módulo: si cambia lo que hace un archivo, actualice su docstring y vuelva a generar esta página con `python docs/src/mapa_codigo.py`.

Otras carpetas: `app/templates/` (páginas Jinja2), `app/static/` (CSS, JS, iconos, logos de marca), `app/locales/` (traducciones), `tests/` (pruebas, capítulo 7), `scripts/` (utilidades de desarrollo), `tools/`, `docs/` (manuales y esta guía) e `install.sh` (instalador y actualizador, capítulo 8).

Núcleo de la aplicación

Archivo	Líneas	Qué hace
`app/main.py`	3852	Aplicación web OpenNCCM (FastAPI).
`app/web.py`	489	Utilidades compartidas por las rutas web.
`app/auth.py`	173	Sesión, roles y alcance:
`app/perms.py`	169	Permisos por usuario:
`app/config.py`	31	Configuración del proceso por variables de entorno (NCM_*):
`app/db.py`	729	Base de datos:
`app/crypto.py`	118	Cifrado AES-256-GCM de credenciales y configuraciones almacenadas.
`app/i18n.py`	300	Idiomas de la interfaz.
`app/scheduler.py`	258	Tareas programadas (cron) y receptor syslog para detección de cambios en tiempo real.
`app/cli.py`	226	Utilidades de línea de comandos.
`app/errors.py`	36	Motivo corto y traducido de un error de conexión o de respaldo (Panel «Necesita tu atención», chat del asistente).
`app/metrics.py`	73	Rendimiento del servidor para el panel:
`app/security.py`	142	Verificación en dos pasos (TOTP), certificado propio del sitio y páginas públicas de privacidad y licencias.
`app/systime.py`	93	Fecha, hora y zona horaria del servidor (timedatectl).
`app/updates.py`	145	Aviso de versiones nuevas y actualización desde la web.
`app/sysbackup.py`	215	Recuperación ante desastres:
`app/__init__.py`	2	OpenNCCM:

Equipos: conexión, respaldo e inventario

Archivo	Líneas	Qué hace
`app/services.py`	1957	Lógica de negocio:
`app/drivers.py`	733	Perfiles de fabricantes.

Archivo	Líneas	Qué hace
`app/shell_drivers.py`	136	Drivers de Netmiko propios para sistemas basados en un shell Unix que Netmiko no trae.
`app/legacy_ssh.py`	83	Compatibilidad SSH con equipos de red antiguos.
`app/fwtag.py`	85	Versión de firmware de cada copia de seguridad (ConfigVersion.fw_version).
`app/chassis.py`	213	Parte de atrás del equipo:
`app/ports.py`	764	Puertos físicos del equipo y su estado (ficha del equipo:
`app/porttable.py`	378	Lectores de la tabla de puertos de cada fabricante (app/ports.py los usa al respaldar).
`app/swos.py`	75	MikroTik SwitchOS (SwOS y SwOS Lite):
`app/fireware_web.py`	196	Feature key de WatchGuard (Fireware) en el formato que se pega para restaurarla.
`app/fxi.py`	772	WatchGuard Fireware:
`app/licenses.py`	127	Vencimiento de los servicios de la licencia guardada de cada equipo (Profile.license_cmd:
`app/bulkbackup.py`	223	«Respaldar todos» (y un grupo o los marcados) con su progreso:
`app/retire.py`	253	Equipos retirados (Dispositivos › Retirados):
`app/restore.py`	222	Restaurar una copia de seguridad (versión de configuración) en otro equipo del mismo modelo.

Cambios, reversión y consola

Archivo	Líneas	Qué hace
`app/rollback.py`	566	Generación de comandos de reversión (current -> target) según la familia de sintaxis.
`app/rollback_vendor.py`	506	Reversión propia de plataformas que no siguen la sintaxis Cisco/Huawei/set:
`app/console.py`	725	Consola SSH web:
`app/consolecenter.py`	838	Operar › Consola:
`app/consolecore.py`	363	Lo que la consola web sabe de cada sistema:
`app/cmdref.py`	423	Comandos de cada equipo para la consola (autocompletado y frecuentes) y para el asistente IA, según su tipo, su modelo y su versión de firmware.
`app/nettools.py`	266	Herramientas de Operar › Consola con parámetros:
`app/favorites.py`	250	Comandos favoritos de la consola (Dispositivos y Operar › Consola).
`app/credview.py`	169	Ver el usuario y la contraseña de la credencial de un equipo.
`app/impact.py`	448	Análisis de impacto:

Recetas, configlets y cumplimiento

Archivo	Líneas	Qué hace
`app/recipes.py`	207	Recetas incluidas en OpenNCCM (Operar › Consola › Recetas):
`app/recipes_more.py`	266	Más recetas incluidas:
`app/recipes_major.py`	371	Recetas de las familias principales:
`app/recipes_fw.py`	315	Recetas de firewalls y routers:
`app/recipes_gen.py`	325	Recetas estándar generadas por familia de CLI:
`app/recipes_rest.py`	351	Recetas de las plataformas que no tenían (firewalls, SD-WAN, transporte, OLT, routers japoneses y otros).
`app/recipes_ubnt_zte.py`	287	Recetas de Ubiquiti y ZTE:
`app/configlets_seed.py`	237	Configlets y scripts incluidos desde 2.34:
`app/configlets_ui.py`	79	Datos de la página Configlets y scripts en tres columnas (como Recetas):
`app/compliance_packs.py`	414	Paquetes de cumplimiento listos para usar:
`app/verified.py`	63	Sello «Verificada» de los configlets y de las correcciones de cumplimiento:
`app/autofix.py`	388	Corrección automática de cumplimiento (opcional, apagada por defecto).
`app/refconfig.py`	587	Configuración de referencia por modelo (Dispositivos › Configuración de referencia).

Red: mapa, vecinos, MAC/IP e informes

Archivo	Líneas	Qué hace
`app/topology.py`	482	Mapa de la red:
`app/topolayout.py`	200	Distribución del Mapa de la red:
`app/nodes.py`	95	Nodos de la red (Administración › Nodos):
`app/nbwatch.py`	459	Aviso de vecino nuevo o perdido (Ajustes › Mapa de la red, Alarmas y el mapa).
`app/macfind.py`	451	Buscar una MAC o una IP:
`app/ipam.py`	157	Inventario de IP y VLAN sacado de las configuraciones guardadas (no se ejecuta nada en los equipos).
`app/mapreport.py`	163	Informes › Equipos del mapa:
`app/netreports.py`	383	Informes › Puertos sin uso y Informes › Documento de la red.
`app/netdoc.py`	624	Documento de la red (Informes › Documento de la red):
`app/reports.py`	218	Informes tabulares (CSV en pantalla, descarga y envío programado por correo) y analítica.
`app/charts.py`	248	Gráficas SVG generadas en el servidor (sin librerías externas).
`app/health.py`	121	Panel:

Firmware, CVE y ciclo de vida

Archivo	Líneas	Qué hace
`app/firmware.py`	288	Actualización remota de firmware:
`app/fwupdates.py`	748	Actualizaciones de firmware de los equipos:
`app/fwverify.py`	157	Versión instalada leída del propio equipo, sin respaldar, y comprobación después de actualizar el firmware.
`app/eol.py`	147	Fin de venta / fin de soporte (EOL/EOS):

Integraciones

Archivo	Líneas	Qué hace
`app/extauth.py`	279	Inicio de sesión con directorios externos:
`app/zabbix.py`	189	Importar equipos desde Zabbix (API JSON-RPC, versiones 6.0 a 7.x).
`app/gitexport.py`	145	Copia de los respaldos en un repositorio Git (GitHub, GitLab, Gitea…):
`app/vendorlogos.py`	70	Logos de marca de los equipos (diseño de Diseño UX, app/vendor_logos.json y static/vendor-logos/*.svg).

Asistente IA y base de conocimiento

Archivo	Líneas	Qué hace
`app/assistant.py`	615	Asistente IA (Administración › Ajustes › Asistente IA):
`app/aichat.py`	763	Chat del asistente en todo el sistema (botón «Asistente» de la barra superior).
`app/kb.py`	1169	Equipo › Base de conocimiento:

Rutas agrupadas y API

Archivo	Líneas	Qué hace
`app/routes_ops.py`	552	Rutas:
`app/mobile.py`	347	API para apps móviles (/api/mobile/…):
`app/mobile_app.py`	714	La app móvil con servidor:

Utilidades

Archivo	Líneas	Qué hace
`app/cronutil.py`	164	Programación de tareas:

Dónde buscar

Quiero cambiar…	Mire en
cómo se respalda un equipo	`services.backup_device` y el `Profile` de su tipo en `drivers.py`
cómo se aplica una solicitud de cambio	`services.execute_change_request` → `apply_config` → `_push_stepwise`
qué comando cuenta como consulta	`consolecore.is_change` (y `_QUERY` / `_DANGER` por familia)
los comandos de «Volver a esta versión»	`rollback.py` (familias) y `rollback_vendor.py` (RouterOS, EXOS, VOSS)
una receta	`recipes*.py` (formato en `recipes.py`) y su sello en `recipes.VERIFIED`
una regla de cumplimiento incluida	`compliance_packs.py`
el panel principal	`main.dashboard` , `reports.analytics` , `health.summary` , `templates/dashboard.html`
la lista de equipos	`main.devices` , `templates/devices.html`
la barra lateral y la superior	`templates/base.html` , `web.render` y `web._nav_counts`
un texto en inglés	`locales/en.json`
lo que ve la app móvil	`mobile_app.py` (pantallas) y `mobile.py` (sesión y acciones)

3. Datos

Todo está en `app/db.py`: el motor, los modelos (SQLAlchemy 2, estilo `Mapped[...]`), la migración y los ajustes.

Motor y sesiones

- **SQLite** por omisión (`DATA_DIR/opennccm.db`), con `journal_mode=WAL` y `foreign_keys=ON` en cada conexión. El grupo de conexiones se dimensiona con `NCM_WORKERS + 8` para que los respaldos en paralelo nunca se queden sin una.
- **PostgreSQL** con `NCM_DATABASE_URL` (el instalador lo ofrece). El código no usa nada propio de un motor salvo en lugares marcados (`engine.dialect.name`): agrupar por fecha en `reports._hour_key` y la copia de recuperación.
- Cada uso abre una sesión corta: `with SessionLocal() as db: ...`. `expire_on_commit=False`: los objetos siguen leíbles después del commit. **Las relaciones se cargan dentro de la sesión** (después da `DetachedInstanceError`); por eso hay líneas como `v.device.name # carga la relación antes de cerrar la sesión`, o mejor, `selectinload`.

Tablas

Tabla	Modelo	Qué guarda
`devices`	`Device`	cada equipo: dirección, tipo (`device_type` = clave del perfil), credencial, grupo, nodo, inventario (modelo, serie, versión), estado del último respaldo, vecinos y puertos (JSON), retiro
`config_versions`	`ConfigVersion`	cada copia de la configuración (`running` o `startup`), **cifrada** (`content_enc`), su sha256, líneas +/−, origen (`manual`, `scheduled`, `syslog`, `push`), firmware de esa copia (`fw_version`)
`credentials`	`Credential`	usuario y contraseña/secreto cifrados, reutilizables entre equipos
`device_groups`, `net_nodes`	`DeviceGroup`, `NetNode`	grupos (permisos, programaciones) y nodos (sitios del mapa)
`users`, `user_groups`, `api_tokens`	`User`, `UserGroup`, `ApiToken`	cuentas locales o de directorio, a qué grupos se limitan y sus tokens de API (solo el hash)
`change_requests`	`ChangeRequest`	solicitudes de cambio: cuerpo (plantilla Jinja2), variables, equipos, estado, resultado, hora programada
`policies`, `rules`, `compliance_results`	`Policy`, `Rule`, `ComplianceResult`	políticas de cumplimiento, sus reglas (regex) y el último resultado de cada regla en cada equipo
`autofix_runs`	`AutoFixRun`	historial de la corrección automática (`app/autofix.py`): equipo, política, reglas, resultado, motivo, salida y versiones de antes y después. La política guarda si está activada y su ventana (`Policy.autofix*`); cada regla elegida, la huella de lo aprobado (`Rule.autofix_sig`)

Tabla	Modelo	Qué guarda
`configlets`, `recipes`	`Configlet`, `Recipe`	plantillas de comandos y recetas propias de los usuarios (las incluidas están en el código)
`schedules`	`Schedule`	programaciones (respaldo, auditoría, CVE, configlet) con su cron
`events`	`Event`	alarmas y eventos (gravedad, categoría, equipo, reconocida)
`audit_log`	`AuditLog`	quién hizo qué y desde dónde
`vulnerabilities`	`Vulnerability`	CVE de NVD por equipo
`eol_entries`	`EolEntry`	fin de venta / soporte por modelo
`firmware_images`, `firmware_updates`	`FirmwareImage`, `FirmwareUpdate`	biblioteca de firmware y última comprobación de versiones nuevas por equipo
`console_sessions`, `console_access_requests`, `console_favorites`	…	sesiones de la consola web (con grabación en disco), pedidos de acceso y comandos favoritos
`kb_*`	`KbCase`, `KbVersion`, …	base de conocimiento: casos, versiones, propuestas, comentarios, votos, «falta documentar»
`mac_seen`, `neighbor_states`	`MacSeen`, `NeighborState`	historial MAC→puerto / IP→MAC y estado de cada vecino LLDP/CDP para avisos
`fxi_images`	`FxiImage`	imágenes .fxi de WatchGuard
`ai_usage`, `ai_chats`	`AiUsage`, `AiChat`	límites y conversaciones del asistente IA
`settings`	`Setting`	ajustes clave → valor (ver abajo)

Migración al actualizar

`init_db()` corre en cada arranque y **solo agrega**:

1. `Base.metadata.create_all(engine)` crea las tablas nuevas.
2. `_add_missing_columns()` compara cada modelo con la tabla real y hace `ALTER TABLE … ADD COLUMN` de lo que falte, con su valor por omisión si es simple (booleano, número o texto).
3. `_add_missing_indexes()` crea los índices declarados en `__table_args__` que no existan (`CREATE INDEX` con `checkfirst`). `create_all` solo crea índices junto con su tabla, por eso hace falta este paso.

Para **agregar una columna**: declárela en el modelo con `default=` (si lo necesita) y listo. Para **agregar un índice**: póngalo en `__table_args__` del modelo con un nombre propio (`ix_<tabla>_<qué>`). Para renombrar o transformar datos no hay mecanismo automático: escriba la migración (una función idempotente llamada desde `init_db`, como `_migrate_themes`) y su prueba en `tests/update-test.sh`.

Nunca borre una columna del modelo en la misma versión que deja de usarla: los servidores que vuelvan a la versión anterior (`rollback-update.sh`) la necesitan.

Ajustes (`settings`)

- `get_setting(clave, por_omisión)` / `set_setting(clave, valor)` . Las claves de `SECRET_SETTINGS` (contraseñas de

SMTP, LDAP, tokens de Git, claves de IA y de NVD…) se guardan cifradas.

- Los ajustes que se leen en cada petición usan `auth.cached_setting` (caché de 30 s; `clear_settings_cache()` al

guardar la página de Ajustes).

- La página **Administración › Ajustes** (`templates/settings.html`) guarda las claves de `SETTING_KEYS` y

`CHECKBOX_KEYS` de `app/main.py` , con sus valores de fábrica en `SETTING_DEFAULTS` . Para un ajuste nuevo: agréguelo ahí, al formulario y, si se lee en cada petición, use `cached_setting` .

Cifrado (`app/crypto.py`)

- `encrypt()` / `decrypt()` : AES-256-GCM con un nonce aleatorio de 12 bytes por valor; se guarda en base64

(nonce + texto cifrado). La clave es `secret.key` (32 bytes; se crea sola la primera vez).

- `decrypt()` lanza `KeyMismatch` si el dato se cifró con otra clave (por ejemplo, una base restaurada sin su

clave): la interfaz lo explica en lugar de fallar. `try_decrypt()` devuelve `None` en ese caso.

- Las contraseñas de usuario son `pbkdf2_sha256` con sal (`hash_password` , `needs_rehash` las actualiza al entrar).
- La cookie de sesión se firma con un secreto derivado de la clave (`session_secret`).

Si se pierde `secret.key` se pierde todo lo cifrado (configuraciones y credenciales). La copia de recuperación (`app/sysbackup.py` , `python -m app.cli system-backup --with-key`) puede incluirla.

Retención y tamaño

- `retention_versions` (Ajustes): cuántas copias guardar por equipo y tipo; la purga corre a las 03:30 y nunca borra

las copias con etiqueta (`label` , p. ej. la línea base).

- Un respaldo igual al anterior no crea copia nueva (se compara el texto normalizado, `services.same_config`): solo

actualiza la fecha del equipo y, si faltaba, el firmware de la última copia.

4. Equipos y plataformas

El perfil de cada tipo de equipo (`app/drivers.py`)

Cada tipo de equipo es un `Profile` en la lista `PROFILES` . `get_profile(device_type)` lo devuelve (`PROFILE_MAP`). Los campos que más se usan:

Campo	Para qué
`key`	el `device_type` de Netmiko (o uno propio registrado en `shell_drivers.py`)
`label` , `vendor`	cómo se muestra
`backup`	comando que da la configuración en ejecución (`show running-config` , `/export` , `show configuration` de Junos en formato «set»…)
`startup`	configuración de arranque (si existe): se compara para avisar de cambios sin guardar
`inventory`	comandos de inventario (uno por línea); `version_re` , `model_re` , `serial_re` sacan versión, modelo y serie
`family`	sintaxis para la reversión: `IOS` , `HUAWEI` , `SET` (Junos, VyOS), `FASTPATH` , `MERGE` (sin reversión por comandos)
`commit` , `save`	si los cambios necesitan `commit` (Junos, PAN-OS, VyOS) o guardar (`write memory`)
`ignore`	regex de líneas volátiles que no cuentan como cambio (fechas, contadores)
`cpe` , `cpe_version` , `cpe_model`	cómo buscar sus CVE en NVD
`firmware`	cómo actualizar el firmware por SCP (sistema de archivos, arranque, recarga)
`license_cmd`	licencia que se lee una vez (WatchGuard)
`backup_timeout`	segundos para leer la configuración (OLT grandes)

El respaldo, paso a paso (`services.backup_device`)

1. Si el equipo está desactivado, no se conecta. Se toma un candado por equipo (`device_lock`): nunca hay dos respaldos del mismo equipo a la vez.
2. `connect(dev)` abre la sesión de Netmiko con su credencial (con los algoritmos SSH antiguos de `legacy_ssh.py` si hace falta). Equipos sin SSH (SwitchOS) van por HTTP (`swos.py`).
3. En **la misma conexión** se lee: la configuración (`backup`), la de arranque (`startup`), el inventario, los vecinos LLDP/CDP (`topology`), los puertos (`ports` / `porttable`), las tablas MAC/ARP (`macfind`) y, si falta, la licencia. Leer todo en una conexión es una regla: cada conexión cuesta segundos y algunos equipos limitan sesiones.
4. `clean_config()` quita lo volátil (`ignore`); se detecta la salida cortada (paginado, falta el `end` final).
5. Si es igual a la última copia (`same_config`), no se crea versión nueva. Si cambió, se guarda cifrada con su diferencia (+/−), su origen y el firmware leído (`fwtag`).
6. Se actualizan inventario, estado, vecinos (y avisos de vecino nuevo/perdido, `nbwatch`), puertos, conflicto con la de arranque y desvío de la línea base. (La diferencia con la configuración de referencia, `refconfig` , se calcula al mirarla.)

7. Después: exportación a Git (`gitexport`), búsqueda de firmware nuevo si el equipo es nuevo (`fwupdates`), auditoría de cumplimiento del equipo y, si fue un cambio no autorizado con la reversión automática encendida, `_auto_rollback` .
8. Si falló, `_check_and_retry` mira si el puerto responde y reintenta una vez; el motivo corto y traducido para la interfaz sale de `errors.short_error` .

Aplicar cambios (`services.execute_change_request` → `apply_config`)

Una solicitud de cambio guarda el cuerpo como **plantilla Jinja2** (variables del usuario y `device.*`). Al ejecutarse, por cada equipo:

1. `render_commands()` arma la lista de comandos (sandbox de Jinja2).
2. `push_commands()` abre una ventana de mantenimiento de 5 minutos (para que el syslog de este cambio no dispare la reversión automática) y llama a `apply_config()` .
3. Red de seguridad del equipo: **MikroTik** en modo seguro (Ctrl+X), **Junos** con `commit confirmed 5` (se vuelve a conectar y se confirma; si el cambio cortó el acceso, el equipo vuelve solo).
4. `_push_stepwise()` manda los comandos **de uno en uno**, lee la respuesta hasta el prompt y se detiene en el primero que el equipo rechace (`DEVICE_ERROR` , una regex con los mensajes de error de cada fabricante) o que pida confirmación (`CONFIRM`). No guarda ni hace commit si algo falló; en plataformas con commit, descarta lo pendiente.
5. Si el comando cambió el nombre del equipo, `_refresh_prompt` actualiza el prompt esperado.
6. Guarda (`save_config`) o hace commit, y al final se respalda el equipo (origen «push»).

Las solicitudes que vienen de una receta, de Multi-equipo o del asistente pasan antes por `consolecenter.strip_wrappers()` , que quita `configure terminal` , `end` , `commit` … porque OpenNCCM ya entra al modo configuración y guarda. **Excepción:** el `end` que cierra un bloque `config …` de FortiOS se conserva, porque en FortiOS es lo que aplica el bloque.

Consultas y cambios en la consola (`app/consolecore.py`)

Multi-equipo, la consola en modo solo lectura y las comprobaciones de las recetas necesitan saber si un comando solo consulta. `os_of(device_type)` da la familia (`ios` , `junos` , `ros` , `fortios` …) y `is_change(os, cmd)` decide con:

- `_QUERY[familia]` : regex de lo que **consulta**. Lo que no coincide **cuenta como cambio**.
- `_DANGER[familia]` : lo que corta el servicio (reinicio, borrar la configuración) y pide confirmación aparte.
- El catálogo de comandos del tipo de equipo (`cmdref.py`): sus consultas conocidas cuentan como consulta.

`PAGING_OFF` (apagar el paginado) y `CONFIG_PROMPT` (reconocer el modo configuración) también van por familia.

Volver a una versión (`app/rollback.py` , `app/rollback_vendor.py`)

OpenNCCM **no** reaplica la configuración entera: calcula la diferencia entre la actual y la de destino y escribe los comandos para deshacerla, que el usuario revisa en una solicitud de cambio.

- `IOS` / `HUAWEI` : bloques por indentación; lo que sobra se niega con `no …` / `undo …` .
- `SET` (Junos, VyOS): `delete …` de lo que sobra y `set …` de lo que falta.
- `FASTPATH` : como IOS con sus modos (`vlan database`).
- Propias (`rollback_vendor.py`): MikroTik RouterOS (`/… remove` , `set`), Extreme EXOS (`unconfigure` , `delete`) y VOSS (`vlan delete` , `no …`).

- `MERGE` (FortiOS, PAN-OS…) y las de `MANUAL`: **sin reversión por comandos**; la interfaz explica cómo restaurar

con la herramienta del fabricante. La reversión automática no se puede activar en esos equipos.

La IP de gestión del equipo nunca se toca al revertir (`mgmt`).

Recetas (`app/recipes*.py`)

Guías paso a paso de Operar › Consola › Recetas. Formato (ver el docstring de `app/recipes.py`):

```
{"key": "ios-ntp", "vendor": "Cisco", "os": "ios", "os_label": "Cisco IOS / IOS-XE", "cat": "servicios",
 "name": N_("Configurar NTP (Cisco IOS)"), "intro": N_("…"), "doc": "https://… (documentación oficial)",
 "vars": [{"n": "ntp", "l": N_("Servidor NTP"), "v": "10.0.0.1"}],
 "steps": [{"t": N_("Servidor NTP."), "c": "configure terminal\nntp server {{ntp}}\nend"},
           {"t": N_("Comprobar."), "c": "show ntp associations", "check": True}]}
```

- Los pasos que cambian algo se aplican con una solicitud de cambio; los de consulta y el de comprobar

(`"check": True`) se ejecutan en el momento por Multi-equipo. Por eso **el comando de comprobación tiene que ser una consulta** para `consolecore` (si no, la comprobación no se puede ejecutar).

- `"warn": True` marca un paso que puede cortar el acceso o pedir confirmación.
- Cada receta se revisa contra la documentación oficial (`doc`).
- **Sello «Verificada»** (`recipes.VERIFIED`): versión en la que se probaron todos sus pasos en un equipo real o en

el laboratorio, con 3 rondas seguidas sin fallos (capítulo 7). Un administrador puede poner o quitar el sello desde la interfaz (ajuste `recipe_verified` , queda en Auditoría).

Configlets y scripts

Plantillas Jinja2 de comandos guardadas en la tabla `configlets` (filtro de tipo de equipo por regex). Los incluidos se cargan con `configlets_seed.py` (`seed_new_configlets` al arrancar agrega los nuevos de cada versión). Su sello «Verificada» está en `app/verified.py` .

Cumplimiento (`app/compliance_packs.py` , `services.run_compliance`)

Una política tiene reglas `must_contain` / `must_not_contain` (regex multilínea sobre la configuración guardada), una gravedad y, opcionalmente, los comandos de corrección. La auditoría corre después de cada respaldo y con las programaciones. Los paquetes incluidos (PCI DSS, HIPAA, SOX, Cisco, MikroTik, Junos, FortiGate…) se cargan al arrancar si faltan.

Cómo agregar una plataforma nueva

Ejemplo: un switch que se maneja como Cisco pero con otro comando de configuración.

1. **Perfil** en `app/drivers.py` (`Profile(...)` en `PROFILES`): `key` de Netmiko, comandos de respaldo e inventario, regex de versión/modelo/serie, familia de reversión, `ignore` , `cpe` . Si Netmiko no lo trae y es un shell Unix con menú, mire `shell_drivers.py` .
2. **Logo** (opcional): la clave de su marca en `app/vendor_logos.json` y el SVG en `app/static/vendor-logos/` ; regenere el sprite con `python scripts/build_vendor_logos.py` .
3. **Familia de la consola** en `consolecore.os_of` y, si no es una ya conocida, su `_QUERY` , `_DANGER` , `PAGING_OFF` y `CONFIG_PROMPT` . Lo que no se defina usa `generic` (seguro: casi todo cuenta como cambio).
4. **Comandos de la consola y del asistente** en `cmdref.py` (autocompletado, frecuentes, consultas conocidas).

5. Si muestra vecinos LLDP/CDP: su comando y su lector en `topology.py`; puertos en `porttable.py`; tablas MAC/ARP en `macfind.py`; IP y VLAN en `ipam.py`. Cada uno es opcional: lo que no se reconoce no cuenta.
6. **Reversión**: si su sintaxis es de una familia conocida, basta la `family`. Si no, una función en `rollback_vendor.py` o `MERGE` (sin reversión por comandos, se explica cómo restaurar).
7. **Recetas** (`recipes_*.py`) y **paquete de cumplimiento** (`compliance_packs.py`), revisados contra la documentación oficial.
8. **Pruebas**: un caso en `tests/unit_vendors.py` (perfil, inventario), lo que agregue en topología/puertos/IPAM en sus `unit_*.py`, y si puede, el laboratorio (capítulo 7) para sellar recetas.
9. **Documentación y registro**: el manual de funciones (`docs/src/manual-funciones.html`) si cambia lo que ve el usuario, `CHANGELOG.md` y las traducciones de los textos nuevos en `app/locales/en.json`.

5. Interfaz web

Rutas

- La mayoría están en `app/main.py` (equipos, versiones, cambios, cumplimiento, usuarios, ajustes…). Las demás en

`APIRouter` propios que `main.py` incluye al final: `routes_ops.py` (firmware, EOL, analítica, recuperación), `consolecenter.py` y `console.py` (consola), `refconfig.py`, `kb.py`, `retire.py`, `mobile.py`, `mobile_app.py`, `security.py`, `assistant.py`, `aichat.py`, etc.

- Las rutas son **funciones normales** (`def`, no `async def`) salvo las que leen un formulario o JSON

(`await request.form()` / `await request.json()`); FastAPI corre las normales en su grupo de hilos.

- Un formulario que cambia algo responde con `go(url)` (redirección 303) y deja el aviso con

`flash(request, texto, "ok"|"error")`.

- Toda acción que importa queda en Auditoría: `log_action(request, user, "accion", "detalle")` (o `audit()`).

Permisos

Rol	Puede
`viewer`	ver
`operator`	además respaldar, auditar, pedir cambios, usar la consola (según Ajustes › Consola)
`admin`	además aprobar cambios, administrar usuarios, credenciales, ajustes y sistema

- `require(request, "operator")` al principio de la ruta. Sin sesión lanza `LoginRequired` (redirige al login);

sin permiso, `Forbidden` (403).

- **Permisos por usuario** (`app/perms.py`, desde 2.39): el rol es una plantilla; cada usuario puede tener su propia

lista de funciones (`User.perms`, JSON; vacío = las de su rol). `require(request, "operator"/"admin")` traduce la ruta a su función con `perms.ROUTES` y comprueba `perms.has(user, clave)`. **Una ruta nueva que pide un rol necesita su línea en `ROUTES`** con una función cuyo rol mínimo (`ROLE_MIN`) sea el mismo: `tests/unit_perms.py` lo comprueba para todas. Dentro de una ruta o una plantilla se pregunta por la función, no por el rol: `perms.has(user, "push")` en Python y `{% if can(user, 'push') %}` en Jinja.

- **Administrador maestro** (`User.is_master`): lo tiene todo, no se puede borrar ni editar por otros y es el único que

da o quita las funciones de `perms.CRITICAL` (`perms.from_form`). Se asigna al primer administrador (`db._ensure_master`) y se recupera con `opennccm set-master USUARIO` (`app/cli.py`).

- **Alcance**: un usuario no admin con grupos asignados solo ve los equipos de esos grupos. `device_scope(user)`

devuelve el conjunto (o `None` = todos); `check_device(user, id)` lanza 403 si no puede ver ese equipo; `filter_ids(user, ids)` filtra una lista. **Toda ruta que recibe un id de equipo tiene que llamar a `check_device`.**

- Quien pide un cambio puede aprobarlo él mismo salvo que se desactive en Ajustes (`allow_self_approval`, separación de funciones).

Plantillas (`app/templates/`)

- Todas heredan de `base.html` (barra lateral, barra superior, avisos, buscador Ctrl K). Las partes reutilizables

empiezan con `_` (`_timeline.html` , `_diffview.html` , `_pager.html` , `_warn.html` …).

- `render(request, "pagina.html", user, **datos)` (`app/web.py`) agrega `user` , `flashes` , `nav_counts` (contadores

de la barra lateral), el aviso de actualización, `profiles` y lo que pase la ruta. Con `compact=True` no calcula los contadores (ventanas emergentes).

- Funciones globales disponibles en todas las plantillas (definidas en `app/web.py`): `icon("nombre")` (sprite

`static/icons.svg`), `vlogo(device_type, tamaño)` (logo de marca), `pl(n, "copia", "copias")` (plurales), `fwpill(...)` (etiqueta de firmware), `spark(...)` (minigráfica), filtros `fecha` y `cuando` («hace 3 min»), `_()` / `_f()` (traducción) y constantes como `ROLES` y `VERSION` .

- Gráficas: se dibujan en el servidor en SVG (`app/charts.py`), con las reglas de diseño del docstring de ese módulo.

Idiomas (`app/i18n.py`)

- Se escribe **en español**. El catálogo `app/locales/en.json` tiene pares `{"texto en español": "traducción"}` .
- En las plantillas **no hace falta marcar los textos**: el cargador (`I18nLoader`) envuelve cada texto visible y

los atributos `placeholder` , `title` , `aria-label` y `alt` . Un texto con valores, «Hay {{ n }} equipos», se busca como «Hay {0} equipos».

- En Python: `_("texto")` , `_f("Respaldo de {0}", nombre)` y `N_("texto")` para marcar textos que se traducen

después (constantes de módulo).

- `python scripts/i18n_missing.py` lista lo que falta traducir; la prueba `e2e_i18n` recorre las páginas en inglés

y falla si queda texto en español. Las entradas nuevas se agregan **al final** de `en.json` .

Archivos estáticos (`app/static/`)

- CSS: `style.css` (base), `senal.css` (diseño «Señal»: panel, tarjetas), `warn.css` , `console.css` , `kb.css` …
- JS propio, sin frameworks: `shell.js` (comportamiento común: menús, diálogos, hojas laterales `data-sheet` ,

listas desplegables, buscador, paginado `data-pager`), `center.js` (consola), `bulkbk.js` (respaldar todos), `topo-layouts.js` (mapa)…

- Las plantillas los enlazan con `?v={{ VERSION }}-{{ HASH }}` ; el servidor los sirve con caché de un año

(`immutable`) cuando llevan `?v=` , así que **cambiar un archivo cambia su hash y el navegador lo vuelve a pedir**. Un estático nuevo enlazado sin `?v=` no se guarda en caché de esa forma.

- Logos de marca: un solo sprite `vendor-logos.svg` generado con `python scripts/build_vendor_logos.py` desde

`static/vendor-logos/*.svg` y `app/vendor_logos.json` (con el texto convertido a trazos). Las versiones chicas son `vl-<marca>-chico` .

Convenciones de la interfaz

- Textos cortos y concretos, en segunda persona, sin jerga innecesaria. Los avisos explican qué pasó y qué hacer.

- Lo peligroso se confirma (`data-confirm`) y lo que corta el servicio se marca como tal.
- Cada pantalla funciona sin JavaScript para lo esencial (formularios normales); el JS mejora la experiencia.
- Accesibilidad: cada campo con su etiqueta, botones de solo símbolo con `aria-label` (`shell.js` completa los que

falten), contraste comprobado por `tests/unit_contrast.py`.

- Temas claro/oscuro y colores de acento: variables CSS; no ponga colores fijos en las plantillas.

6. API y app móvil

API REST con token (`/api/…`)

Para scripts e integraciones. Cada usuario crea sus tokens en **Mi cuenta › Tokens de API** (empiezan con `ncm_` ; solo se guarda su sha256, con vencimiento opcional). Se envían en la cabecera:

```
curl -H "Authorization: Bearer ncm_…" https://opennccm.ejemplo/api/devices
```

El token actúa **con los permisos y grupos de su dueño** (`auth._token_user`). Solo vale en rutas que empiezan con `/api/` . Sin token válido la respuesta es `401` en JSON.

Método y ruta	Qué hace
`GET /api/devices`	lista de equipos (los que el dueño del token puede ver)
`POST /api/devices`	dar de alta un equipo
`GET /api/devices/{id}`	un equipo con su inventario y estado
`GET /api/devices/{id}/versions`	sus versiones
`GET /api/devices/{id}/config`	la última configuración (texto)
`POST /api/devices/{id}/backup`	respaldar ahora (encola el trabajo)
`GET /api/config-search?q=…`	buscar texto en las configuraciones

Otras rutas `/api/…` las usa la propia interfaz (consola, mapa, buscar MAC/IP, asistente, favoritos) y responden JSON con la sesión del navegador; no son una API pública estable.

Para agregar una ruta a la API pública: póngala bajo `/api/` , use `require(request, rol)` (vale con sesión o token), filtre por `device_scope` , devuelva JSON y agréguela a `tests/e2e_api.py` y a esta tabla.

API de la app móvil (`/api/mobile/…`)

La app móvil se conecta al servidor con la dirección, usuario y contraseña (locales o de Active Directory; con verificación en dos pasos si la cuenta la tiene) y recibe un token propio.

- `app/mobile.py` : sesión (`/api/mobile/login` , `logout` , `me`) y acciones básicas (resumen, equipos, respaldar, versiones, alarmas, aprobar cambios, avisos).
- `app/mobile_app.py` : **una ruta JSON por pantalla**, que lee lo mismo que su página web (`/api/mobile/panel` , `devlist` , `device/{id}` , `diff` , `alarms` , `configlets` , `compliance` , `vulns` , `firmware` , `analytics` , `map` , `ipam` , `reports` , `kb` , `admin` , `account` …). Si cambia lo que muestra una página web, revise su par móvil.
- Consola por WebSocket: `/api/mobile/devices/{id}/console/ws` (mismo protocolo v2 que la consola web).

El puente de acciones (`MobileBridge`)

Para no duplicar la lógica de cada acción (aprobar un cambio, crear un caso, dar de alta un equipo…), la app llama a **la misma ruta web** a través de `/api/mobile/act/<ruta web>` :

1. `MobileBridge` (middleware ASGI) comprueba que la ruta esté en la lista permitida `ACT` (método + expresión).

2. Lee el token de la app de `Authorization: Bearer …` y pone el usuario en `scope["ncm_mobile_uid"]`. `auth.current_user()` lo mira primero, así que la ruta web ve a ese usuario como si tuviera sesión.
3. Llama a la ruta web y convierte la respuesta (redirección + avisos) en JSON para la app.

Para que la app pueda usar una acción web nueva: agréguela a `ACT` (lo más ajustada posible) y a `tests/e2e_mobile_app.py`.

Seguridad de las API

- Todo lo que se puede hacer por API se puede hacer en la web con el mismo rol: no hay atajos.
- Las rutas que cambian algo exigen `operator` o `admin` igual que en la web, y quedan en Auditoría.
- Los tokens se revocan desde Mi cuenta › Tokens de API.

7. Pruebas y laboratorio

Hay tres niveles de prueba. Los dos primeros corren solos en cada cambio; el tercero se hace a mano cuando se quiere dar por **verificada** una receta o una plataforma.

1. Pruebas automáticas (`tests/`)

- `unit_*.py` : sin red ni servidor. Prueban una función o un módulo (lectores de inventario, puertos, topología, reversión, clasificación de comandos…). Se ejecutan con `python tests/unit_x.py` .
- `e2e_*.py` : contra un OpenNCCM real en `127.0.0.1:8000` y el **Cisco simulado** (`tests/fakedev.py` , SSH en el puerto 2222; el 2223 con `--legacy` ofrece solo ssh-rsa y kex SHA-1). Recorren la web como un usuario (con `requests`) y revisan el HTML, la base de datos o la API.
- Todas siguen el mismo molde: `check(nombre, condición, detalle)` imprime `OK` o `FAIL` , y al final el script sale con código 1 si algo falló.

```
bash tests/run-tests.sh .venv/bin/python   # crea un directorio de datos temporal, el admin, arranca el
                                           # servidor y los dos equipos simulados, y corre la lista
completa
```

Para correr una sola prueba e2e, arranque el servidor como hace `run-tests.sh` (variables `NCM_DATA_DIR` , `NCM_KEY_FILE` , `NCM_SYSLOG_PORT=5514` , `NCM_FW_CHECK_NEW=0` ; admin `admin` / `Admin12345` ; `python -m app.cli seed`) y luego `python tests/e2e_x.py` . Algunas e2e dependen de otras anteriores (por ejemplo `e2e_legacy` usa la credencial que crea `e2e`): respete el orden de la lista.

Una prueba nueva se agrega al final de la lista de `tests/run-tests.sh` . Cada arreglo de un error lleva una prueba que fallaba antes del arreglo.

`tests/fakedev.py` simula varios sistemas con opciones (`--edgeswitch` , `--exos` , `--shell=pfsense|opnsense|…` , `--enable-pass=X`): mire su docstring antes de escribir un simulador nuevo.

2. Integración continua (`.github/workflows/ci.yml`)

En cada *push* y *pull request*, GitHub Actions corre (15 trabajos):

Trabajo	Qué prueba
Revisión de código	`shellcheck` de los scripts y `pyflakes` de todo el Python
Aplicación (Ubuntu 22.04 y 24.04)	`tests/run-tests.sh` , OpenSSH antiguo real y restaurar una copia con PostgreSQL
Directorios reales	Samba AD (LDAP) y Keycloak (OpenID Connect)
Servidores Linux reales	respaldar Ubuntu 26.04/24.04/22.04 y Debian 13 por SSH (contenedores oficiales)
MikroTik real	RouterOS CHR en QEMU/KVM: cambio no autorizado, reversión en modo seguro, fallo a mitad
Zabbix real	importar equipos desde Zabbix 6 y 7

Trabajo	Qué prueba
Instalación y actualización	`install.sh` desatendido, certificado propio, nginx, zona horaria, actualización desde la versión anterior y vuelta atrás

Una versión no se publica si algún trabajo falla. Hay dos pruebas que a veces fallan por tiempos en las máquinas de GitHub (`unit_db_pool` en Ubuntu 22.04 y `e2e_console2`): si fallan solas, vuelva a correr solo los trabajos fallidos antes de buscar un error.

3. Laboratorio con equipos virtuales (GNS3) y el sello «Verificada»

Para afirmar que una receta, un configlet o una función funciona **en un sistema de verdad**, se prueba en el laboratorio: máquinas virtuales de los fabricantes en GNS3, manejadas por un OpenNCCM de prueba como lo haría un usuario (por la web y la API), sin tocar equipos de producción.

Cómo está armado

- Un proyecto de GNS3 con una **nube** conectada a la red del laboratorio, un switch de gestión y, por plataforma,

dos equipos (`X-1` principal y `X-2`) unidos por dos puertos de datos (para «restaurar en otro», el mapa por LLDP y el aviso de vecino perdido). La gestión de cada equipo sale por DHCP de la red del laboratorio.

- Un **OpenNCCM de prueba** en otro puerto (con su propio directorio de datos) que recibe el syslog de los equipos.
- Un script de **primer arranque** por plataforma que entra por la consola de GNS3 (telnet), pone la contraseña del

laboratorio, el nombre, la gestión por DHCP y SSH, y muestra la IP.

Las etapas

Cada ronda corre las mismas etapas en cada plataforma, y anota cada comprobación (OK/FALLA con su detalle):

Etapa	Qué se comprueba
01 Alta, respaldo e inventario	respaldo completo, versión, modelo y serie; un segundo respaldo igual no crea versión
02 Firmware	«Comprobar ahora» lee la versión del propio equipo
04 Consola web	abrir la consola SSH en el navegador, ejecutar un comando y que quede grabada
05 Multi-equipo	una consulta en los dos a la vez; un cambio no se ejecuta directo (pide solicitud)
06 Configlet con variables	solicitud con variables, aprobación, aplicación y el respaldo de después
07 Cambiar el nombre	el prompt cambia y la solicitud termina bien
08 Configuración grande	decenas de comandos en una solicitud y el respaldo los tiene todos
09 Comando inválido	la solicitud queda como fallida (no «bien»)
10 Syslog	un cambio hecho por fuera dispara un respaldo de origen «syslog»
11 Reversión automática	un cambio no autorizado se quita solo
12 Volver a una versión	la comparación lo muestra y la reversión lo quita
13 Borrar y restaurar	borrar el equipo, volverlo gestionable y restaurar la copia en texto

Etapa	Qué se comprueba
14 Restaurar en otro	clonar la copia de X-1 en X-2 (mismo modelo)
15 Configuración de referencia	encontrar diferencias y prepararlas para igualar
16 CVE	buscar vulnerabilidades de esa versión
17-18 Mapa y vecino perdido	el enlace entre los dos por LLDP/CDP; apagar los puertos da la alarma y al volver se cierra
20 Buscar MAC o IP	dónde se ve una MAC/IP
21 Puertos	la franja de puertos de la ficha
22 Cumplimiento	el paquete de la plataforma se aplica
23 Recetas	**cada receta de la plataforma**: todos sus pasos por solicitud y su comprobación por Multi-equipo; después se deshace lo que hizo

Lo que una plataforma no admite por diseño se anota como «no aplica» con su motivo, y se comprueba que OpenNCCM lo explique (por ejemplo, FortiOS no tiene reversión por comandos: se verifica que «Volver a esta versión» lo diga y no arme comandos).

El sello

Una receta (o configlet, o corrección de cumplimiento) recibe el sello **«Verificada · <sistema> <versión>»** cuando pasa **3 rondas seguidas sin fallos** en esa versión. Se escribe en `recipes.VERIFIED` (o `verified.py`) con un comentario de dónde y cuándo se probó:

```
# Juniper Junos 14.1R1.10 (vMX en GNS3), 3 rondas seguidas sin fallos, 2026-11-09
"junos-desc": "Junos 14.1R1.10", ...
```

Una falla de la prueba (no del producto) no cuenta: se arregla la prueba y se vuelven a correr 3 rondas. Una falla del producto se arregla en OpenNCCM, con su prueba automática (`tests/unit_*.py`) para que no vuelva, y también se repiten las 3 rondas.

Lo que el laboratorio ya encontró (y quedó como prueba automática)

- Junos: la serie sale de `show chassis hardware`; un vMX sin serie no toma «VMX» como serie (`unit_lab_multi`).
- OPNsense: el aviso de entrada llega en trozos y no se confunde con el prompt (`unit_lab_multi`).
- RouterOS 6: la pantalla de avance de `/certificate sign` no es el prompt (`unit_ros_progress`).
- FortiOS: el `end` de los bloques `config …` no se quita de las solicitudes; `diagnose sys ntp status` y `diagnose sniffer packet` son consultas; la configuración de referencia no arma comandos al estilo Cisco (`unit_wrappers`).
- Junos (vQFX): `family inet dhcp vendor-id …` se borra desde `dhcp` al revertir; y un commit que falla descarta la configuración candidata con `rollback 0`, porque en Junos es compartida y el cambio siguiente la arrastraba (`unit_junos_commit`).

Reglas del laboratorio

- Solo equipos virtuales o de prueba del laboratorio, nunca de un cliente.

- La contraseña del laboratorio vive en un archivo fuera del repositorio; los scripts no la imprimen.
- Lo que cada prueba cambia se deshace al terminar (la siguiente ronda empieza igual).
- Algunas imágenes necesitan funciones del hipervisor: vJunos-switch exige KVM anidado («Nested KVM is not enabled,

shutting down»); el vQFX necesita su RE (máquina clásica, adaptadores e1000) y su PFE unidos por `em1`.

8. Publicar y actualizar

Versiones

- `MAYOR.MENOR.PARCHE` en `app/__init__.py` (`__version__`). El instalador y `opennccm update` la leen de ahí con

`grep` : deje la línea `__version__ = "X.Y.Z"` tal cual.

- MENOR sube con funciones nuevas; PARCHE con arreglos. Cada versión junta varios cambios (no se publica una versión

por cada arreglo pequeño), salvo un arreglo urgente de seguridad o de algo que impide trabajar.

CHANGELOG.md

Una sección `## X.Y.Z` al principio, y una línea por cambio, en palabras del usuario (qué ve o qué puede hacer), no del código:

```
## 2.37.0
- [nuevo] **Título corto:** qué hace y dónde está (Menú › Página).
- [mejora] **…:** …
- [arreglo] **…:** qué fallaba, en qué equipo o versión, y qué hace ahora.
- [seguridad] **…:** …
```

Las notas de la versión en GitHub se arman solas con esa sección.

Cómo se publica

1. Rama nueva desde `main` (por ejemplo `proxima-version-38`) con los cambios, sus pruebas y su línea en `CHANGELOG.md` .
2. Suba `__version__` , escriba la sección del CHANGELOG y, si cambió lo que ve el usuario, actualice el manual (`docs/src/manual-funciones.html` ; capturas con `docs/src/capturas.py`) y regenere los PDF con `python docs/src/build.py` (usa Microsoft Edge en modo sin ventana).
3. *Pull request* a `main` . Espere a que pasen **todos** los trabajos de la CI (capítulo 7) y únala.
4. Cree la etiqueta sobre `main` y súbala:

`bash git checkout main && git pull git tag v2.37.0 && git push origin v2.37.0`

1. La CI vuelve a correr todo con la etiqueta y, si pasa, el trabajo «Publicar versión» crea la versión en GitHub con las notas del CHANGELOG, el `.tar.gz` del código y los PDF de `docs/` .

Cómo se actualizan los servidores

- `sudo opennccm update` (o desde la web: Administración › Sistema › Actualizar, que deja un archivo

`update.request` que atiende `opennccm-webupdate.path` como root).

- El actualizador (`scripts/opennccm.sh` , instalado como `/usr/local/bin/opennccm`): hace una copia de la base y del

código, trae las etiquetas de GitHub, cambia a la versión nueva, instala las dependencias, reaplica la configuración del sistema (`install.sh` en modo actualización: servicio, nginx, polkit) y reinicia. Al arrancar, `init_db()` migra la base (capítulo 3).

- `sudo openniccm rollback` vuelve a la versión anterior con su base de datos.
- `--version vX.Y.Z` instala una versión concreta; `--channel main` la rama de desarrollo (solo para pruebas).

`tests/update-test.sh` (en la CI) instala la versión anterior, crea datos, actualiza a la nueva, comprueba que todo sigue ahí y prueba la vuelta atrás. Si una versión cambia algo del instalador o de la base, esa prueba tiene que seguir pasando.

El instalador (`install.sh`)

Un solo script para instalar (`curl … | sudo bash` , interactivo o `--unattended`) y para actualizar. Instala los paquetes del sistema, crea el usuario `openniccm` , el entorno de Python, la clave, el servicio de systemd, nginx con HTTPS (certificado propio, de Let's Encrypt o uno suyo), el cortafuegos (si se pide) y el primer administrador. Toda opción interactiva tiene su parámetro para el modo desatendido. Pase `shellcheck` antes de subir un cambio (la CI lo exige).

Documentación para el usuario

Archivo	Fuente
`docs/OpenNCCM-Guia-de-instalacion.pdf`	`docs/src/guia-instalacion.html`
`docs/OpenNCCM-Manual-de-funciones.pdf`	`docs/src/manual-funciones.html`
`docs/OpenNCCM-Guia-del-desarrollador.pdf`	esta guía (`docs/desarrollo/*.md`), con `python docs/src/guia_dev.py`
`README.md` y `docs/readme/*.png`	capturas con `docs/src/readme_capturas.py`

9. Rendimiento

OpenNCCM tiene que seguir siendo rápido con miles de equipos y decenas de miles de copias. Casi todo lo lento que se encontró venía de la base de datos, no de Python ni de las plantillas. Estas son las reglas.

Reglas

1. **No traiga objetos enteros para contar.** `db.scalars(select(ConfigVersion))` carga cada copia **con su texto cifrado**. Para contar o agrupar, pida solo las columnas (`select(ConfigVersion.device_id, ConfigVersion.created_at)`) o, mejor, cuente en la base (`func.count` , `group_by`). Ejemplo: `reports.analytics` agrupa por hora y origen en SQL (`_hour_key` , compatible con SQLite y PostgreSQL) en lugar de recorrer 20 000 copias.
2. **Nada de una consulta por equipo dentro de un bucle** (N+1). Para «la última copia de cada equipo» use `services.latest_versions(db, ids)` (una consulta con `max(id)` agrupado). Para relaciones, `selectinload` o `joinedload` en la misma consulta.
3. **Filtre en Python o en la plantilla solo lo que ya es chico.** Un `selectattr` sobre todas las fallas, dentro de un bucle por regla, es cuadrático: prepare un diccionario en la ruta (`fails_by_rule`) y búsquelo en la plantilla.
4. **No lea columnas grandes que la página no muestra.** La lista de equipos difiere (`defer`) inventario, vecinos, puertos y licencia (`main.LIST_DEFER`). Cuidado: un atributo diferido que se lee después de cerrar la sesión da error, así que solo se difiere lo que la plantilla no usa.
5. **Índices que cubran la consulta.** En SQLite, si la columna que filtra está después de una columna de texto grande, leer la fila entera es lo caro: un índice con todas las columnas que usa la consulta (`ix_cv_kind_created` , `ix_cv_device_kind` , `ix_events_ack_id`) la resuelve sin tocar la tabla. Antes de agregar un índice, mire el plan con `EXPLAIN QUERY PLAN` **de todas las consultas de esa tabla**: un índice nuevo puede hacer que SQLite elija mal en otra (pasó con las alarmas: un índice por `acknowledged, severity` hizo que «las 5 más recientes» ordenara 13 000 filas; se cambió a `acknowledged, id, …` y las listas ordenan por `id`).
6. **Lo que se parsea de una copia guardada se puede guardar en memoria por id de versión** (no cambia nunca): `main._ipam_parse` guarda las IP y VLAN de cada copia.
7. **Las respuestas grandes van comprimidas** (`_Gzip` en `main.py`) y los estáticos con `?v=` se guardan un año en el navegador. Una respuesta que llega en vivo línea a línea (como ping) no se comprime: agréguela a `_Gzip.LIVE` .
8. **Lo que se calcula en cada página** (la barra lateral: `web._nav_counts`) tiene que ser una consulta barata con índice; mídalo con la base grande.

Cómo medir

Base grande de prueba (2000 equipos, 20 000 copias, 60 000 eventos; direcciones de la red de pruebas 198.18.0.0/15):

```
export NCM_DATA_DIR=/tmp/perf NCM_KEY_FILE=/tmp/perf/secret.key NCM_SYSLOG_ENABLED=0 NCM_FW_CHECK_NEW=0
mkdir -p /tmp/perf && python scripts/perf_seed.py
uvicorn app.main:app --port 8006 &
python scripts/perf_bench.py 8006
```

Para comparar con la versión anterior: otra copia del código (por ejemplo `git worktree add /tmp/anterior v2.36.0`) con **una copia de la base sin los índices nuevos**, en otro puerto, y `python scripts/perf_bench.py 8006 8007` : alterna las peticiones entre los dos para que la carga de la máquina afecte igual a ambos.

Para ver **dónde** se va el tiempo de una página: `cProfile` alrededor de la llamada a la ruta (o de la función: `reports.analytics(None, 30)`), ordenado por tiempo acumulado, y filtrado por `app/`. Si lo que más tarda es `fetchall` o `loading.py` de SQLAlchemy, es la regla 1, 2 o 5.

Resultado de la versión 2.37 (2000 equipos, mismo equipo y misma carga)

Página	Antes	Después	Por la red
Panel (`/`)	1422 ms	404 ms	41 KB → 10 KB
Dispositivos	403 ms	394 ms	3975 KB → 208 KB
Cumplimiento	1086 ms	347 ms	3226 KB → 74 KB
Búsqueda en configuraciones	790 ms	221 ms	32 KB → 8 KB
IP y VLAN	1741 ms	65 ms	32 KB → 8 KB
Analítica	1008 ms	76 ms	45 KB → 10 KB
Alarmas	53 ms	26 ms	44 KB → 9 KB
Ficha de un equipo	37 ms	24 ms	77 KB → 15 KB
App móvil: panel	471 ms	198 ms	—
App móvil: lista de equipos	144 ms	153 ms	961 KB → 50 KB

Lo que queda más lento es dibujar la lista de 2000 equipos (unos 400 ms, casi todo en la plantilla). Si hiciera falta, el siguiente paso sería paginarla en el servidor (hoy se pagina en el navegador con `data-pager`).